

```
satya@elgence: ~/Desktop/Article/DanglingPointers
satya
vi case2.c
satya
cc -Wall case2.c
case2.o: In function 'main':
case2.c:15:7: warning: variable 'iptr' set but not used [-Wunused-but-set-variable]
int *iptr;
```

Figure 3: Compiler generating warning

```
12
13 //Some statements
14
15 *block_ptr = 20;
16 //Pointer becomes dangling, since the memory
17 //block to which it is pointing is already freed
18
19 return 0;
20 }
```

In the code snippet shown above,

Line 5: Memory allocation by `malloc()`.

Line 9: Memory allocated is freed by `free()` manually.

Line 13: Reusing the pointer, which is still pointing to the memory that is already freed. In our example, `block_ptr` is now the dangling pointer.



Note: In Case 1: Memory is freed automatically.

In Case 3: Memory is freed manually. This is one of the key differences between stack and heap.

In the C programming language, deleting an object from memory explicitly or by destroying the stack frame on the return of the control does not alter associated pointers as seen in Case 1 and Case 3. The pointer still points to the same location in memory, even though the reference has since been deleted and may now be used for other purposes.

Solution for the problem in Case 3: In Case 3, the

dangling pointer can be avoided by initialising it to NULL immediately after freeing it, if the OS is capable of detecting the runtime references as shown below:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main()
5 {
6     int *block_ptr = (int *) malloc(sizeof(int));
7
8     //Do something with allocated memory
9     free(block_ptr);
10    //Initialising the block pointer to NULL
11    block_ptr = NULL;
12    //Now, block_ptr is no more dangling
13    //Some statements
14
15    *block_ptr = 20;
16    //Error: Dereferencing the NULL pointer, gives
17    //segmentation fault
18    return 0;
19 }
```

Dangling pointers are very harmful and have adverse effects in embedded systems programming. So, they should be strictly avoided. **END**

By: Satyanarayana Sampangi

The author is a member of the embedded software team at Emeritxe Information Technology (P) Ltd (<http://www.emeritxe.com>). His areas of interest are embedded C programming combined with data structures and microcontrollers. He can be reached at satya@emeritxe.com

Read more stories on Security in
www.electronicb2b.com

TOP SECURITY STORIES

- CCTV camera market is expected to double by 2015
- The latest in biometric devices
- CCTV Camera manufacturers can look forward to a bright future
- Video Analytics Systems Turning a CCTV into a proactive tool
- Security cameras evolving with new technologies
- The latest in dome cameras
- The latest in weather-proof and vandal-resistant security cameras

**ELECTRONICS
B2B
INDUSTRY IS AT A**

CLICK OF A BUTTON

Login to www.electronicb2b.com and be in touch with the Electronics B2B Fraternity 24x7